🧾 Project Documentation

---

1. 🚀 Project Overview

Full Stack CFO is an intelligent financial decision-support system designed for small businesses. It goes beyond expense tracking by actively determining **what to pay, when to pay, and why**, while also assisting with communication and identifying growth opportunities.

The system transforms fragmented financial inputs into **structured decisions, actionable outputs, and strategic insights**, effectively acting as a virtual Chief Financial Officer.

---

2. 🎯 Problem Statement

Small businesses struggle with:

- Managing multiple types of financial obligations (rent, salaries, vendors, taxes, etc.)
- Deciding payment priorities under limited cash flow
- Understanding trade-offs between penalties, relationships, and survival
- Communicating with vendors during delays
- Utilizing surplus capital effectively

Existing tools:

- Track finances ❌
- Automate bookkeeping ❌
- But **do not make decisions** ❌

---

3. 💡 Solution

Full Stack CFO introduces a **decision-first financial system** that:

- Aggregates all financial obligations
- Uses deterministic logic to prioritize payments
- Generates multiple payment strategies
- Explains decisions in simple language
- Automates vendor communication

- Suggests business opportunities when surplus exists

---

4. 🧠 Core Principles

4.1 Deterministic + AI Hybrid Architecture

- **Python Engine** → All financial calculations and decisions
- **LLM (Gemini API)** → Language, explanation, and reasoning

AI never performs arithmetic — ensuring reliability and trust.

4.2 Obligation-Centric Model

Instead of “bills,” the system models **all business obligations**, including:

- Fixed costs (salary, rent, tax, EMI)
- Operational costs (utilities, subscriptions)
- Vendor payments
- Discretionary spending

4.3 Explainability First

Every decision is:

- Transparent
- Justified
- Human-readable

---

5. 🏗️ System Architecture

5.1 Authentication Layer

- OAuth (Google / GitHub)
- Stores:
    - User identity

    - Business domain (used for opportunity suggestions)

5.2 Data Ingestion Layer

Input Sources:

- OCR (receipts via Tesseract)
- PDF invoices (PyMuPDF)
- Bank statements (CSV via pandas)
- Manual entry (UI forms)

Processing:

- OCR output cleaned using Gemini API
- Converted into structured JSON format

5.3 Data Storage Layer

Core Table: `obligations`

Field	Description
id	Unique identifier
user_id	Owner
amount	Payment value
due_date	Deadline
obligation_type	Category
flexibility_tier	Delay flexibility
vendor_id	Linked vendor (optional)
penalty_pct	Late penalty

Obligation Types

- salary
- rent
- tax

- loan_emi
- utility
- subscription
- vendor
- other

Flexibility Tiers

Tier	Meaning	Examples
0	Non-negotiable	salary, tax, rent, EMI
1	Low flexibility	utilities
2	Medium	vendors
3	High flexibility	discretionary

5.4 Decision Engine (Python)

This is the **core intelligence layer**.

Step 1: Runway Calculation

Determines how long the business can survive:

days_to_zero = balance / avg_daily_outflow

Step 2: Priority Scoring

Tier 0 Handling:

- Always paid first
- Sorted by due date

Tier 1–3 Scoring Factors:

- Penalty severity
- Urgency (due date proximity)
- Vendor relationship history
- Small bill bonus (clears multiple small dues first)

Step 3: Vendor Intelligence

Tracks:

- Payment reliability
- Disputes
- Relationship duration

Influences prioritization scores.

Step 4: Conflict Detection

If:

total_obligations > available_balance

→ System triggers scenario generation

5.5 Scenario Engine

Generates multiple payment strategies:

Examples:

- Penalty minimization strategy
- Relationship preservation strategy
- Runway maximization strategy

Each scenario is evaluated based on:

- Financial penalties
- Survival duration
- Relationship impact

5.6 AI Layer (Gemini API)

Used strictly for non-deterministic tasks:

---

1. OCR Cleanup

Converts raw text → structured financial data

---

2. Email Generation

Creates vendor communication:

- Payment deferrals
- Negotiation messages
- Tone-aware (formal/friendly)

---

3. Decision Explanation

Explains:

- Why payments are ordered a certain way
- What trade-offs were made

---

4. Opportunity Engine

Triggered when:

surplus > threshold

Outputs:

- 3 actionable business opportunities
- Expected ROI
- Effort level
- Time to impact

5.7 Output Layer

- Payment schedule
- Scenario comparisons
- Email drafts
- PDF export (reportlab / WeasyPrint)

5.8 Frontend (React)

Dashboard Features:

- Runway visualization
- Obligation breakdown by category
- Scenario selection cards
- Email preview panel
- Opportunity suggestions

---

6. 🔄 End-to-End Workflow

1. User logs in
2. Inputs or uploads financial data
3. Data is normalized
4. Decision engine:
    - Calculates runway
    - Scores obligations
    - Detects conflicts
5. Scenario engine generates strategies (if needed)
6. Best scenario selected
7. AI layer:
    - Explains decision
    - Drafts emails
8. If surplus:
    - Suggests opportunities
9. Outputs displayed and exported

---

7. 🌟 Key Innovations

7.1 Small Bill Prioritization

Automatically clears multiple small debts first without explicit rules.

7.2 Vendor Relationship Scoring

Adds real-world business intelligence to financial decisions.

7.3 Full Obligation Modeling

Moves beyond invoices → models entire business cost structure.

7.4 Scenario-Based Decision Making

Transforms system from calculator → strategic advisor.

7.5 Surplus Opportunity Engine

Shifts focus from survival → growth.

---

8. 🧪 Evaluation Criteria Alignment

Criteria	How Addressed
Data robustness	Multi-source ingestion + normalization
Decision integrity	Deterministic Python logic
Strategic reasoning	Scenario engine
Explainability	AI-generated explanations
Usability	Clean dashboard + automation

---

9. 🛠️ Tech Stack

Layer	Technology
Backend	Python (pandas, logic engine)
AI	Gemini API
OCR	Tesseract + OpenCV
Database	SQLite / Supabase

Frontend	React + Recharts
Auth	OAuth (Google/GitHub)
PDF	reportlab / WeasyPrint

---

10. 📈 Future Enhancements

- Real-time bank integrations
- Predictive cash flow forecasting
- Automated payment execution
- Credit risk scoring
- Multi-user (team finance collaboration)

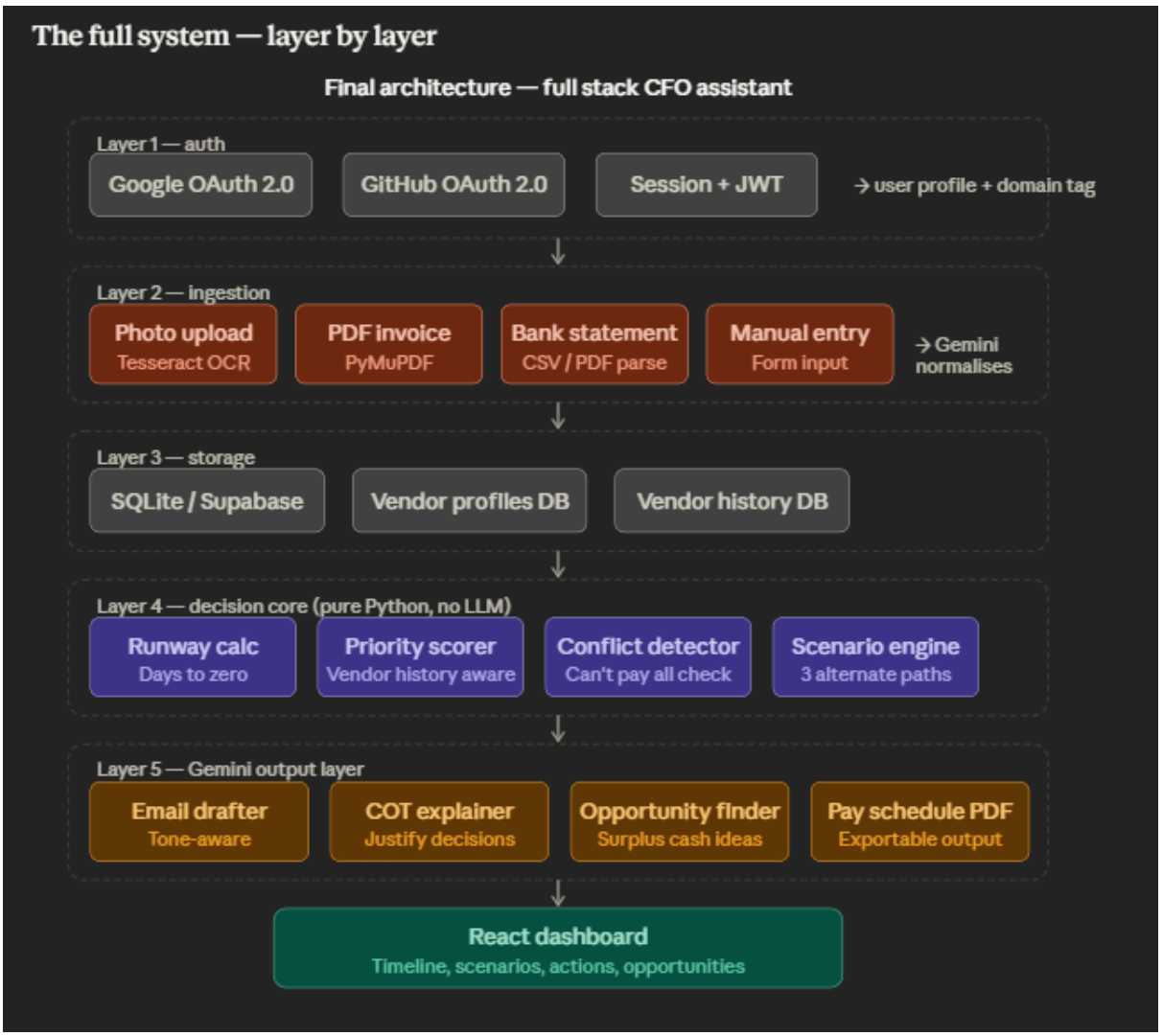
---

11. 🧨 Final Summary

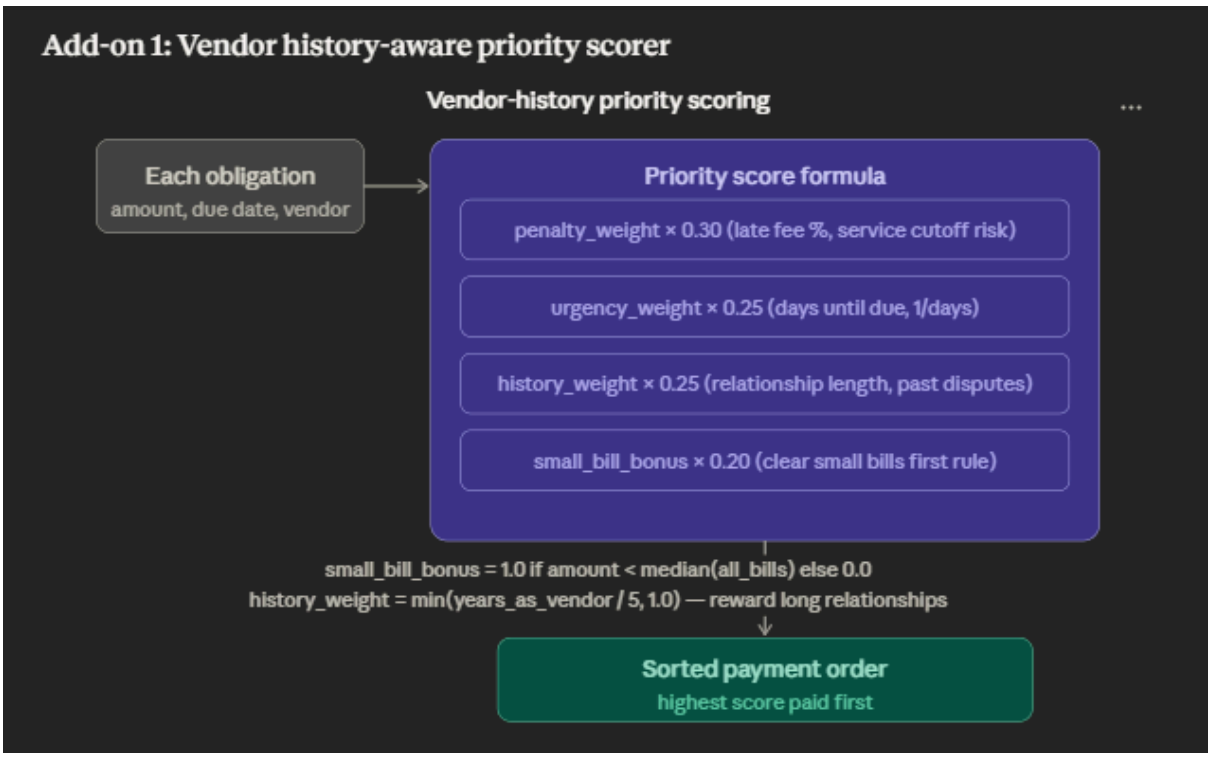
Full Stack CFO is not just a financial tracker — it is a **decision-making system** that:

- Prioritizes payments intelligently
- Explains financial strategy clearly
- Automates communication
- Identifies growth opportunities

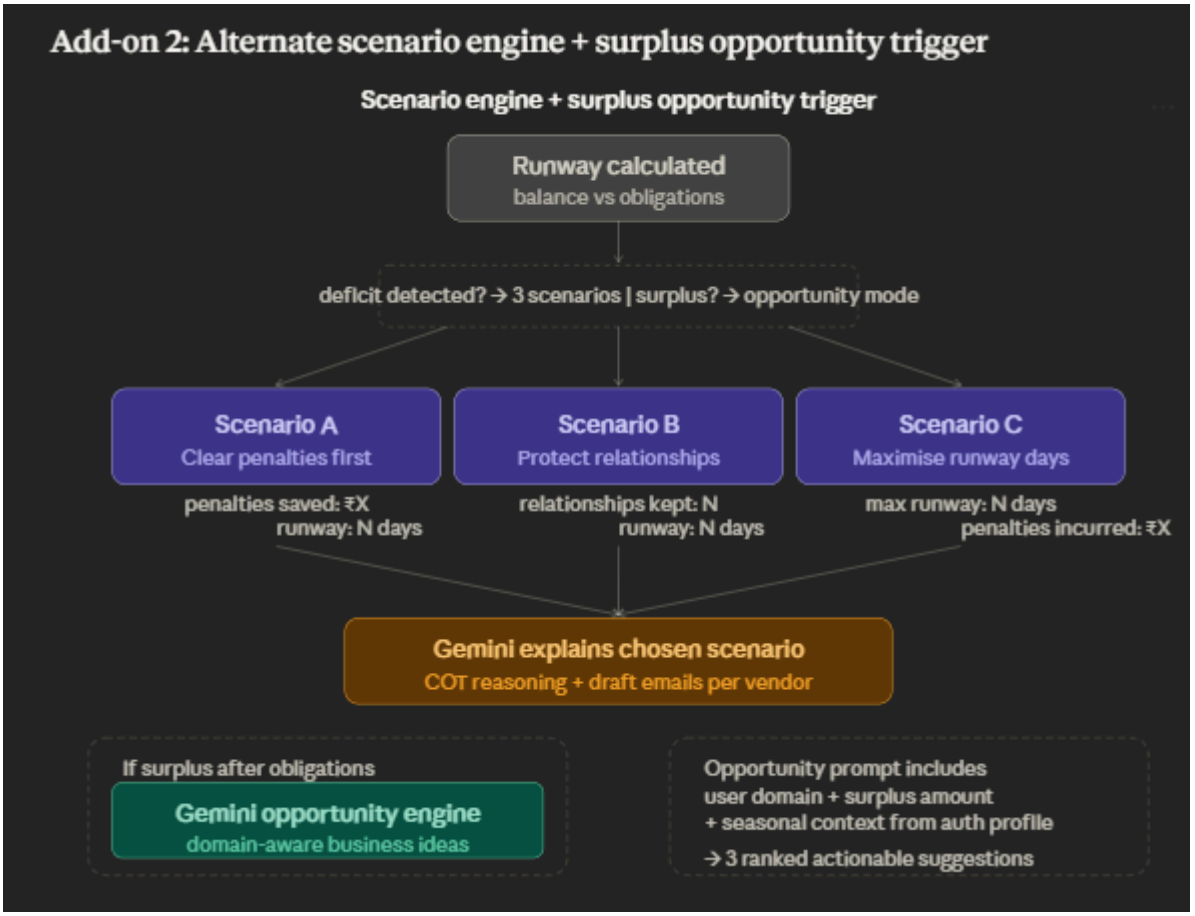
It bridges the gap between **financial data and business action**, enabling small businesses to operate with the intelligence of a CFO.



PRIORITY FORMULA



SCENARIO CHOOSER



🧩 System Component Table (Refined)

Layer	Component	Technology	Notes
Auth	Login Page	Google + GitHub OAuth	Use `authlib` (Python) or `NextAuth` (Next.js). Capture & store **user domain** on first login for personalization
Ingestion	Photo Receipts	Tesseract (`pytesseract`) + OpenCV	Apply preprocessing: grayscale, thresholding, skew correction
Ingestion	PDF Invoices	PyMuPDF	Support both digital + scanned PDFs
Ingestion	Bank Statements	pandas (CSV) / PyMuPDF	Detect headers dynamically, handle merged/misaligned columns
Ingestion	Normalization	Gemini API	Prompt → *"Extract {amount, date, vendor, type} as JSON from raw text"*

🗄️ Storage Layer

Component	Technology	Schema / Notes
Transactions	SQLite / Supabase	`transactions(id, user_id, amount, due_date, vendor_id, type)`
Vendor Profiles	SQLite	`vendors(id, name, relationship_years, flexibility, tone_profile)`
Vendor History	SQLite	`vendor_history(id, vendor_id, paid_on_time, disputes, last_contacted)`

🧠 Decision Engine

Component	Technology	Notes
Runway Calculator	Python / pandas	`days_to_zero = balance / avg_daily_outflow`
Priority Scorer	Python	Weighted scoring (due date, penalties, vendor relationship, amount)
Conflict Detector	Python	Trigger if `weekly_obligations > balance`
Small Bill Rule	Python	Add `small_bill_bonus` weight
Scenario Engine	Python	Generate multiple payment sequences → evaluate tradeoffs

📤 Output Layer

Component	Technology	Notes
Email Drafter	Gemini API	Inject vendor tone + reasoning into prompt
COT Explainer	Gemini API	Explain payment decisions in simple terms
Opportunity Finder	Gemini API	Trigger only when surplus > threshold
Pay Schedule PDF	reportlab / WeasyPrint	Generate downloadable structured plan

🖥️ Frontend

Component	Technology	Notes
Dashboard	React + Recharts	Timeline chart, scenario cards, email preview, opportunity insights